

Arbeitsanweisung

Praktikum 2 - Entwicklungsmodelle

Arbeitsanweisung

Die Aufgabe: "Vom Wasserfall zum Sprint"

Die Rahmenbedingungen:

Es werden kleine Teams (5–6 Personen) gebildet.

Die Aufgabe ist die Entwicklung eines kleinen Konsolen-basierten Tools (ein "Lagerverwaltungssystem").

Phase 1: Das Wasserfall-Modell (Die starre Welt)

Der Auftrag: Das Team erhält ein fertiges, detailliertes Lastenheft.

Die Regel: Es darf während der Implementierung keine Rückfrage beim "Kunden" (Tutor/Dozent) gestellt werden.

Ziel: Design erstellen, Code schreiben, am Ende abgeben.

Phase 2: Scrum / Agile Entwicklung (Die flexible Welt)

Der Auftrag: Ein ähnliches Projekt (), aber diesmal in zwei Sprints à 30–45 Minuten .

Die Regel: Es gibt ein Product Backlog (User Stories).

Nach dem ersten Sprint gibt es ein Review mit dem Kunden (Dozent).

Der Kunde darf hier aktiv Feedback geben und Prioritäten ändern.

*1 Der Reflexionsteil (Der wichtigste Part):

Nach der praktischen Arbeit müssen die Teams einen kurzen Bericht erstellen, der folgende Fragen beantwortet:

*1 Reflexion quasi:

Struktur: In welchem Modell habt ihr euch "sicherer" gefühlt?

Fehler: Wann habt ihr gemerkt, dass ihr am Kunden vorbei entwickelt habt?

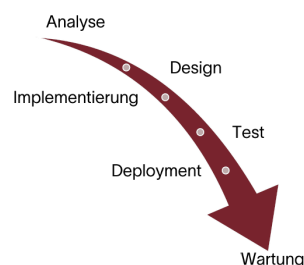
Transfer: Für welche Art von Projekten würdet ihr welches Modell wählen?

EXPORT als JSON

und die Labels doch in Englisch

WASSERFALL-MODELL

- Ein linear-sequenzielles Modell, bei dem jede Phase abgeschlossen sein muss, bevor die nächste beginnt.
- Der Fortschritt fließt wie ein Wasserfall in eine Richtung – zurückgehen zu vorherigen Phasen ist schwierig und kostspielig.
- Starker Fokus auf umfassende Dokumentation am Ende jeder Phase als Grundlage für die nächste.
- Klare Trennung der Verantwortlichkeiten zwischen den einzelnen Phasen.



Phase 1, Phase 2 und Reflexiion

Phase 1

Das Programm soll ein einfaches Lagerverwaltungssystem auf der Konsole sein.

- Es soll Artikel mit Name, Bestand und Mindestbestand speichern.
- Es soll alle Artikel anzeigen können.
- Es soll den Bestand eines Artikels erhöhen können.
- Es soll den Bestand eines Artikels verringern können.

Der Bestand darf nicht negativ werden.

Wenn der Bestand unter den Mindestbestand fällt, soll eine Warnung erscheinen.

Das Programm wird in Java umgesetzt und über ein Zahlenmenü bedient.

Start

- Menü anzeigen
- Eingabe lesen
 - 1 -> Artikel anlegen
 - 2 -> Artikel anzeigen
 - 3 -> Bestand erhöhen
 - 4 -> Bestand verringern
 - prüfen: negativer Bestand?
 - ja -> Fehlermeldung
 - nein -> Bestand speichern
 - prüfen: unter Minimum?
 - ja -> Warnung
 - 0 -> Programm beenden

Ende

Code im Anhang oder:

<https://github.com/Ph4ntomic/MIS-Praktikum-VomWasserfallZumSprint>

Phase 2:

Ziel:

Das Lagerverwaltungssystem wird in zwei kurzen Sprints entwickelt.

Die Anforderungen stehen im Product Backlog als User Stories.

Nach Sprint 1 gibt der Kunde Feedback und darf Prioritäten ändern.

Rollen:

Product Owner: Dozent/Kunde

Scrum Master: achtet auf Ablauf und Zeit

Entwicklungsteam: programmiert und testet

Product Backlog:

US-01: Artikel anlegen und löschen

US-02: Bestand erhöhen und verringern

US-03: Warnung bei niedrigem Bestand

US-04: Daten als JSON speichern und laden

US-05: Englische Labels verwenden

Sprint 1:

Ziel: Grundsystem erstellen

Umsetzung:

Artikel anlegen

Artikel löschen

Bestand erhöhen

Bestand verringern

Negative Bestände verhindern

Ergebnis:

Ein erstes lauffähiges Konsolenprogramm ist vorhanden.

Review nach Sprint 1:

Der Kunde testet das System und gibt Feedback.

Feedback:

Keine CSV-Speicherung.

Stattdessen JSON verwenden.

Deutsche Labels durch englische Labels ersetzen.

Warnfunktion bleibt wichtig.

Sprint 2:

Ziel: Kundenfeedback einarbeiten

Umsetzung:

JSON-Speicherung

JSON-Laden

Warnung bei Mindestbestand

Englische Begriffe im Menü

Fazit:

Scrum war hier sinnvoll, weil Änderungen nach dem Kundenfeedback direkt umgesetzt werden konnten.

Besonders JSON statt CSV und englische Labels zeigen den Vorteil gegenüber dem starren Wasserfallmodell.

Code im Anhang oder:

<https://github.com/Ph4ntomic/MIS-Praktikum-VomWasserfallZumSprint>

Reflexion:

Struktur: In welchem Modell habt ihr euch sicherer gefühlt?

Im Wasserfallmodell haben wir uns am Anfang sicherer gefühlt, weil das Lastenheft fest war und wir einen klaren Plan hatten. In Scrum war es flexibler, aber auch etwas unsicherer, weil sich Anforderungen ändern konnten.

Fehler: Wann habt ihr gemerkt, dass ihr am Kunden vorbei entwickelt habt?

Wir haben es gemerkt, als kein Kundenkontakt während der Entwicklung möglich war. Dadurch konnten wir nicht direkt prüfen, ob unsere Umsetzung wirklich den Erwartungen entspricht. Spätere Änderungen zeigten das deutlich: Der Export sollte statt CSV plötzlich JSON sein und deutsche Begriffe wie Artikelname, Menge, Preis und Mindestbestand sollten auf englische Labels wie name, quantity, price und minimum geändert werden.

Transfer: Für welche Art von Projekten würdet ihr welches Modell wählen?

Das Wasserfallmodell würden wir für kleine, klare Projekte mit stabilen Anforderungen wählen.

Scrum würden wir für Projekte wählen, bei denen sich Anforderungen ändern können oder regelmäßiges Kundenfeedback wichtig ist. Unser Beispiel mit CSV zu JSON und deutschen zu englischen Labels zeigt, dass solche Änderungen in Scrum besser aufgenommen werden können.

Bei sehr kostspieligen oder komplexen Projekten würden wir kein reines Wasserfallmodell wählen, weil Änderungen spät teuer werden können und das Zurückgehen im Prozess schwierig ist.

LagerMini.java

```
import java.util.Scanner;

public class LagerMini {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] id = new String[10];
        String[] name = new String[10];
        int[] quantity = new int[10];
        int[] minimum = new int[10];
        double[] price = new double[10];

        int count = 0;

        while (true) {
            System.out.println("\n1 Add | 2 Show | 3 Plus | 4 Minus | 5
Export JSON | 0 Exit");
            int choice = sc.nextInt();
            sc.nextLine();

            if (choice == 0) {
                break;
            }

            if (choice == 1) {
                if (count >= 10) {
                    System.out.println("Error: Storage is full.");
                    continue;
                }

                System.out.print("ID: ");
                id[count] = sc.nextLine();

                System.out.print("Name: ");
                name[count] = sc.nextLine();

                System.out.print("Quantity: ");
                quantity[count] = sc.nextInt();

                System.out.print("Minimum: ");
                minimum[count] = sc.nextInt();

                System.out.print("Price: ");
```



```

        price[count] = sc.nextDouble();
        sc.nextLine();

        count++;
    }

    if (choice == 2) {
        for (int i = 0; i < count; i++) {
            System.out.println(
                i + " | ID: " + id[i]
                + " | Name: " + name[i]
                + " | Quantity: " + quantity[i]
                + " | Minimum: " + minimum[i]
                + " | Price: " + price[i]
            );
        }
    }

    if (choice == 3) {
        System.out.print("Item number: ");
        int number = sc.nextInt();

        System.out.print("Amount: ");
        int amount = sc.nextInt();

        quantity[number] = quantity[number] + amount;
    }

    if (choice == 4) {
        System.out.print("Item number: ");
        int number = sc.nextInt();

        System.out.print("Amount: ");
        int amount = sc.nextInt();

        if (quantity[number] - amount < 0) {
            System.out.println("Error: Quantity cannot be
negative.");
        } else {
            quantity[number] = quantity[number] - amount;

            if (quantity[number] < minimum[number]) {

```

```
        System.out.println("Warning: Minimum quantity  
reached.");  
    }  
}  
  
    if (choice == 5) {  
        ExportJson.exportJson(id, name, quantity, minimum,  
price, count);  
    }  
}  
  
    sc.close();  
    System.out.println("Program ended.");  
}  
}
```

ExportJson.java

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class ExportJson {

    public static void exportJson(
        String[] id,
        String[] name,
        int[] quantity,
        int[] minimum,
        double[] price,
        int count
    ) {
        try (PrintWriter writer = new PrintWriter(new
FileWriter("warehouse.json"))) {

            writer.println("[");

            for (int i = 0; i < count; i++) {
                writer.println("    {");
                writer.println("        \"id\": \"" + id[i] + "\",");
                writer.println("        \"name\": \"" + name[i] + "\",");
                writer.println("        \"quantity\": " + quantity[i] +
",");

                writer.println("        \"minimum\": " + minimum[i] + ",");
                writer.println("        \"price\": " + price[i]);
                writer.print("    }");

                if (i < count - 1) {
                    writer.println(",");
                } else {
                    writer.println();
                }
            }

            writer.println("]");

            System.out.println("JSON export completed:
warehouse.json");

        } catch (IOException e) {
            System.out.println("Error while exporting JSON.");
        }
    }
}
```

```
}  
}  
}
```